Elements of AIML Lab
Experiment 06 (After-submission Reference): Missing Values and Outliers

Tofik Ali

March 1, 2026

Contents

1 How to use this reference **1**

2 Missing values: what they mean **1**

3 Outliers: error vs rare-but-real **2**

4 Worked example (end-to-end) **2**
4.1 Step 1: load dataset and summarize missingness 2
4.2 Step 2: choose a missing-value strategy (with justification) 2
4.3 Step 3: detect outliers (IQR rule) 2
4.4 Step 4: handle outliers (two common options) 3
4.5 Step 5: compare before/after impact 3

5 What strong observations look like (examples) **3**

6 Troubleshooting **3**

1 How to use this reference

This reference is meant to be shared **after you submit** Experiment 06. It explains **why** missing values and outliers appear, and shows common handling strategies with clear trade-offs.

2 Missing values: what they mean

Missing values are not all the same:

- **MCAR** (missing completely at random): missingness is unrelated to any variable.
- **MAR** (missing at random): missingness depends on observed variables (e.g., income missing more often for certain job types).
- **MNAR** (missing not at random): missingness depends on the missing value itself (hardest case).

In practice, you rarely prove the type; you justify a reasonable strategy and document it.

3 Outliers: error vs rare-but-real

Outliers can be:

- **errors** (wrong unit, extra zero, data entry mistake), or
- **rare-but-real events** (fraud, extreme weather).

Therefore, deleting outliers without reasoning can damage the analysis.

4 Worked example (end-to-end)

4.1 Step 1: load dataset and summarize missingness

```
import pandas as pd

df = pd.read_csv("processed.csv") # or your dataset

missing_count = df.isna().sum().sort_values(ascending=False)
missing_pct = (df.isna().mean() * 100).sort_values(ascending=False)

print(missing_count.head(10))
print(missing_pct.head(10))
```

4.2 Step 2: choose a missing-value strategy (with justification)

Option A: dropping. Use when missingness is small and dropping does not remove important groups.

```
df_drop = df.dropna()
```

Option B: simple imputation (typical baseline).

```
df_imp = df.copy()

num_cols = df_imp.select_dtypes("number").columns
df_imp[num_cols] = df_imp[num_cols].fillna(df_imp[num_cols].median())

cat_cols = df_imp.select_dtypes(exclude="number").columns
for c in cat_cols:
    df_imp[c] = df_imp[c].fillna(df_imp[c].mode().iloc[0])
```

Option C: scikit-learn imputers (pipeline-friendly). This is useful when you later combine preprocessing + modeling.

```
from sklearn.impute import SimpleImputer

imp_median = SimpleImputer(strategy="median")
```

4.3 Step 3: detect outliers (IQR rule)

For one numeric column `num_col`:

```
import numpy as np

x = df_imp["num_col"] # choose a numeric column
q1, q3 = x.quantile(0.25), x.quantile(0.75)
```

```
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr

mask = (x < lower) | (x > upper)
print("Outliers by IQR:", mask.sum())
```

4.4 Step 4: handle outliers (two common options)

Option A: remove outlier rows (only if justified).

```
df_no_out = df_imp.loc[~mask].copy()
```

Option B: cap outliers (winsorization).

```
df_cap = df_imp.copy()
df_cap["num_col"] = df_cap["num_col"].clip(lower, upper)
```

4.5 Step 5: compare before/after impact

```
print("Before:\n", df.describe())
print("After imputation:\n", df_imp.describe())
print("After capping:\n", df_cap.describe())
```

Add one plot if possible (boxplot before vs after) to show the effect clearly.

5 What strong observations look like (examples)

- "`Age` had 4.2% missing values. We used median imputation because the distribution is skewed."
- "IQR detected 12 extreme values in `Income`. We capped them to reduce distortion while keeping rare cases."
- "After capping, max income reduced from 9.8M to 1.2M and mean decreased, indicating extreme values were pulling the mean upward."

6 Troubleshooting

- **All values flagged as outliers:** check if the column is categorical coded as numbers, or if units are inconsistent.
- **Imputation changes categories incorrectly:** avoid mode fill when the column has many unique IDs.
- **Leakage warning (for ML):** always fit imputers/outlier rules on training data only when doing model evaluation.

Sources: Provost and Fawcett, *Data Science for Business.*;McKinney, *Python for Data Analysis.*;WEKA Documentation (Waikato Environment for Knowledge Analysis), accessed 2026-02-28.;Matplotlib and Seaborn documentation, accessed 2026-02-28.